

INSTRUCCIONES DE INSTALACIÓN

Aparecen en el apartado “README” de nuestro proyecto. Igualmente las detallamos en este documento.

Ejecución con Docker

Requisito:

Docker Desktop

Desde la raíz del proyecto:

```
docker compose up --build
```

Servicios disponibles:

Backend: `http://localhost:8000`
Swagger: `http://localhost:8000/docs`
Grafana: `http://localhost:3001`
PostgreSQL Docker: `localhost:5433`

Credenciales de Grafana:

Usuario: `admin`
Password: `1234`

La base de datos se inicializa automáticamente con:

```
db/iot_db_backup.sql
```

Grafana carga automáticamente:

`grafana/dashboard-dryers.json`
`grafana/dashboard-scales.json`
`grafana/dashboard-summary-kpis.json`

Para parar los contenedores:

```
docker compose down
```

Para borrar también los volúmenes y reiniciar la base desde el backup:

```
docker compose down -v
```

Ejecución local sin Docker

Base de datos

Base de datos utilizada:

```
iot_db
```

Tablas principales:

```
devices
telemetry
alarms
scale_events
```

Backup SQL:

```
db/iot_db_backup.sql
```

Para importarlo en pgAdmin:

1. Crear una base de datos llamada `iot_db`.
2. Abrir Query Tool.
3. Ejecutar el contenido de `db/iot_db_backup.sql`.

Backend

Desde la raíz del proyecto:

```
cd C:\xampp\htdocs\Practicas\iot-backend
venv\Scripts\activate
python -m uvicorn main:app --reload
```

La API queda disponible en:

```
http://127.0.0.1:8000
```

Swagger:

```
http://127.0.0.1:8000/docs
```

Simulador

Con el backend encendido, abrir otra terminal:

```
cd C:\xampp\htdocs\Practicas\iot-backend
venv\Scripts\activate
cd iot-telemetry-project-simulator
python -m src
```

El simulador envía datos a:

```
POST /telemetry/
POST /scale-events/
```

Configuración PostgreSQL

La conexión está en:

```
app/core/database.py
```

Por defecto usa:

```
host: localhost
database: iot_db
user: postgres
password: 1234
```

En Docker se configura mediante variables de entorno:

```
DB_HOST
DB_PORT
DB_NAME
DB_USER
DB_PASSWORD
```